

Evidence Pack for Platform Terms 7.e.i.2 Response

Meta App ID 1655751472297844 (auto-publishing) · Threads account @sasohan.insight · Prepared 2026-09-14 · Second submission (first: 2026-08-25)

Purpose of this document. Our first response (2026-08-25) described the changes we made but attached no evidence. This pack provides (1) the exact code that now gates every Threads publish call, (2) the live configuration, (3) a complete log of every Threads-bound pipeline run since the changes were deployed, showing that not a single post was published without passing the gate, and (4) the git commit that can be used to verify all of it.

이 문서의 목적. 2026-08-25 첫 응답에는 변경 내용만 있고 증빙이 없었습니다. 이 문서는 (1) 모든 Threads 발행이 통과하는 게이트 코드, (2) 실제 운영 설정, (3) 변경 배포 이후 Threads 대상 파이프라인의 전체 실행 기록(게이트를 거치지 않고 발행된 게시물도 0건임을 보여줌), (4) 검증용 git 커밋을 담고 있습니다.

1. Summary of facts

App / account	Meta App 1655751472297844 ("auto-publishing") · Threads App ID 1637029624216135 · single account @sasohan.insight, operated by one individual
API used	Official Threads Graph API only (graph.threads.net/v1.0; threads_basic, threads_content_publish). No scraping, no unofficial endpoints, no browser automation against Threads.
Restriction detected	2026-08-18 — every publish call returned OAuthException code 200 "API access deactivated"
Remediation deployed	2026-08-25 13:29 KST — git commit 71b8241 (§6), plus schedule/config changes in .env (§3)
Posts published to Threads since 2026-08-18	0 (last post: 2026-08-17 19:01:12 KST)
Threads-bound pipeline runs after remediation (2026-08-25 → 2026-09-14)	30 runs — backlink_to_sns: 14, coupang_to_threads: 16
Blocked by quality gate (body too short)	28 posts
Not published — no operator approval within 10 min	14 posts (fail-closed: silence = do not publish)
Passed gate, then rejected by Meta (app deactivated)	2 post
No Threads attempt (source fetch failed first)	0 runs
Posts that bypassed the gate	0 — the gate is the only code path to the Threads API (§4)
Discontinued pipelines	newspick_to_threads (third-party news) and aliexpress_to_threads: zero runs after 2026-08-25 (schedules removed)

2. Timeline

Date (KST)	Event
2026-05-14 → 2026-08-17	473 posts published via the API (monthly — 2026-05: 78, 2026-06: 79, 2026-07: 199, 2026-08: 117). Daily average 5.8, maximum 14, against the Threads API limit of 250/day.
2026-08-18	App API access deactivated by Meta. All subsequent publish calls fail; no post has been published since 2026-08-17.
2026-08-25 13:29	Remediation committed and deployed (git 71b8241): publish gate (quality + volume + human approval), fallback-publishing paths removed, schedules cut from 11 to 2 slots/day.
2026-08-25 13:35	First response submitted in the Meta App Dashboard.
2026-08-25 → 2026-09-14	Gate in continuous operation; every Threads-bound run logged (§5). Result: 0 posts published, 0 posts bypassing the gate.
2026-09-14	First response marked "not resolved". This evidence pack prepared for resubmission.

3. Current configuration (live .env, secrets redacted)

3.1 Posting schedule — before vs. after

Pipeline	Before 2026-08-25	Now
coupang_to_threads (product posts, affiliate)	07:30, 12:30, 17:30, 19:00 (4/day)	12:30 (1/day)
newspick_to_threads (third-party news articles)	08:00, 11:30, 14:00, 20:00 (4/day)	discontinued
backlink_to_sns (links to the operator's own blog posts)	13:00, 21:00 (2/day)	21:00 (1/day)
aliexpress_to_threads (overseas products, affiliate)	16:00 (1/day)	discontinued
Total automated slots	11/day	2/day (–82%)
THREADS_MODE	single	single (3-reply "chain" expansion disabled; replies also pass the gate)

```
# ———— 활성화 (수익화 직결, 자격 검증 완료) ————
SCHEDULE_NEWSPICK_TISTORY=09:00,18:00      # 뉴스픽 → 티스토리 (추천인 어필리에이트)
# 네이버 RabbitWrite 'no privilege' 차단으로 비활성 (Playwright 우회 시 재활성)
SCHEDULE_NEWSPICK_NAVER=
# [2026-08-25] Meta 플랫폼 약관 7.e.i.2 시정 - 뉴스픽 Threads 발행 중단.
# 제3자 언론사 기사 기반 게시라 재유통 성격이 강하고, AI 캡션 실패 시 원문
# 제목(베트남어/영어)이 그대로 나가 한국어 계정의 언어 불일치를 만들었다.
SCHEDULE_NEWSPICK_THREADS=                  # (중단) 이전: 08:00,11:30,14:00,20:00
SCHEDULE_COUPANG_TISTORY=08:30,14:30      # 쿠팡 → 티스토리 (2건/일)
# 네이버 RabbitWrite 'no privilege' 차단으로 비활성 (Playwright 우회 구현 시 재활성)
SCHEDULE_COUPANG_NAVER_BLOG=
# [2026-08-25] Meta 플랫폼 약관 7.e.i.2 시정 - 4회 → 1회.
SCHEDULE_COUPANG_THREADS=12:30            # 쿠팡 → Threads (이전: 07:30,12:30,17:30,19:00)
# Threads 장기 토큰 자동 갱신 - Meta Graph API 60일 만료 전 새벽에 갱신
SCHEDULE_THREADS_REFRESH=03:00            # common/threads_token.py

# ———— 비활성 (자격 미준비 또는 정보성 - 활성화 시 시각 입력) ————
# 빈 값으로 두려면 인라인 `# 코멘트` 금지. dotenv 는 unquoted 빈 값 뒤
# 인라인 코멘트를 값의 일부로 파싱하므로 비활성 라인엔 코멘트를 붙이지 말 것.
SCHEDULE_NEWSPICK_WP=
SCHEDULE_NEWSPICK_SNS=
SCHEDULE_COUPANG_WP=
SCHEDULE_COUPANG_GITHUB=
```

```

SCHEDULE_COUPANG_PINTEREST=
SCHEDULE_REALESTATE=
# [2026-08-25] Meta 플랫폼 약관 7.e.i.2 시정 - 2회 → 1회.
SCHEDULE_BACKLINK_SNS=21:00 # 매일 1회 (이전: 13:00,21:00)
SCHEDULE_RISESET_NAVER=
SCHEDULE_RISESET_TISTORY=
SCHEDULE_POLICY_TISTORY=
SCHEDULE_ALIEXPRESS_GITHUB=
SCHEDULE_ALIEXPRESS_WP=
SCHEDULE_ALIEXPRESS_NAVER_BLOG=
# [2026-08-25] Meta 플랫폼 약관 7.e.i.2 시정 - 알리 Threads 발행 중단.
SCHEDULE_ALIEXPRESS_THREADS= # (중단) 이전: 16:00
SCHEDULE_ALIEXPRESS_TISTORY=12:00,17:00 # 알리 → 티스토리 (2건/일, 13:00 backlink 충돌 회피)

```

3.2 Gate parameters

Parameter	Value	Effect
THREADS_MAX_PER_DAY	3	hard cap on total posts per day (the schedule itself only allows 2)
THREADS_MAX_AFFILIATE_PER_DAY	1	at most one affiliate-link post per day
THREADS_MAX_AFFILIATE_RATIO	0.30	affiliate posts ≤ 30% of posts in a rolling 30-day window
THREADS_MIN_CHARS	80	substantive body (excluding disclosure line, URLs, hashtags, emoji) must be ≥ 80 chars
THREADS_REQUIRE_KOREAN / MIN_KOREAN_RATIO	true / 0.30	posts not in the account's language (Korean) are blocked
THREADS_APPROVAL_REQUIRED / TIMEOUT	true / 600 s	every post is sent to the operator via Telegram; only an explicit approval reply publishes; no reply = not published

```

THREADS_MODE=single # single (기본) | chain (3편 reply chain)

# — 발행 게이트 (Meta 플랫폼 약관 7.e.i.2 시정, 2026-08-25) —————
# 2026-08-18 Meta 가 앱 API 접근을 차단했다 ("API access deactivated", code 200).
# 실측 473건(05-14~08-17) 분석에서 드러난 개연성 높은 원인 3가지에 대한 조치다:
# 1. 자동 게시 슬롯 하루 11회 → 위 SCHEDULE_* 로 2회로 감축
# 2. 제휴 링크 게시물 비중 57.3% → 아래 상한으로 제한
# 3. AI 생성 실패 시 폴백 발행 → 품질 게이트 + 파이프라인 폴백 제거
#
# 게이트는 publishers/threads.py 진입점에 있어 모든 Threads 발행이 반드시
# 통과한다. 상세: docs/meta-threads-appeal-2026-08.md
# 현황 확인: python -m common.threads_guard
THREADS_GUARD_ENABLED=true # 게이트 전체 on/off
THREADS_MAX_PER_DAY=3 # 하루 총 게시물 상한
THREADS_MAX_AFFILIATE_PER_DAY=1 # 제휴 게시물 하루 상한
THREADS_MAX_AFFILIATE_RATIO=0.30 # 최근 30일 제휴 비중 상한
THREADS_MIN_CHARS=80 # 실질 본문 최소 길이(고지문·URL 제외)
THREADS_REQUIRE_KOREAN=true # 계정 주 언어 외 게시물 차단
THREADS_MIN_KOREAN_RATIO=0.30 # 한글 최소 비율

# — 발행 사전 승인 —————
# 무인 자동 게시를 중단하고, 모든 게시물을 텔레그램으로 받아 승인한 건만 발행.
# 답글로 승인(ok/승인/o) 또는 거부(그 외 텍스트). 무응답은 발행하지 않는다.
# 승인 응답 수신은 tistory_bridge 의 텔레그램 long-poll 스레드가 처리하므로
# 스케줄러가 떠 있어야 한다(TISTORY_PUBLISHER=bridge 면 자동 임베드).
THREADS_APPROVAL_REQUIRED=true # 승인 게이트 on/off
THREADS_APPROVAL_TIMEOUT=600 # 승인 대기 초 (초과 시 미발행)

```

4. How the gate works

Single choke point. The gate is implemented inside `publishers/threads.py` → `ThreadsPublisher.post()` and `.reply()`, the only two methods in the codebase that call the Threads

API. Every pipeline (product, backlink, or any future one) must go through them. Order of checks, all before any API call:

1. **Quality** (`common/threads_guard.check_quality`): substantive body length ≥ 80 chars; Korean ratio $\geq 30\%$. This closes the path that previously let one-line product names and untranslated foreign-language article titles through when AI caption generation failed. Those fallback paths were also deleted from the pipelines themselves (Appendix C).
2. **Volume** (`check_quota`): daily total cap, daily affiliate cap, rolling 30-day affiliate ratio cap. Counters are persisted on disk.
3. **Human approval** (`common/threads_approval.gate`): the full post text is sent to the operator on Telegram with a forced-reply prompt. Only an exact approval keyword publishes. Any other reply rejects. No reply within 10 minutes rejects. If the Telegram message cannot be sent, the post is rejected — there is no path that falls back to unattended publishing.
4. Only then: create container → publish.

Fail-closed design. If the gate modules cannot even be imported, `_gate()` returns "do not publish". There is no runtime override; disabling the gate requires editing code or config on the operator's own machine.

5. Activity log after remediation (2026-08-25 21:00:16 → 2026-09-14 12:30:23 KST)

Source: data/pipeline_runs.json (live) merged with the daily backups in backups/*.zip. Identical to the attached CSV. Timestamps are pipeline start times; each row is one publish attempt, or one run with no attempt. "Published?" is NO for every row.

Started (KST)	Pipeline	Outcome	Detail	Published?
2026-08-25 21:00:16	backlink_to_sns	BLOCKED_BY_QUALITY_GATE	substantive body 42 chars < minimum 80 chars	NO
2026-08-25 21:00:16	backlink_to_sns	BLOCKED_BY_QUALITY_GATE	substantive body 43 chars < minimum 80 chars	NO
2026-08-26 12:30:07	coupang_to_threads	NOT_PUBLISHED_NO_OPERATOR_APPROVAL	no operator approval within 10 min -> not published (fail-closed)	NO
2026-08-26 21:00:22	backlink_to_sns	BLOCKED_BY_QUALITY_GATE	substantive body 42 chars < minimum 80 chars	NO
2026-08-26 21:00:22	backlink_to_sns	BLOCKED_BY_QUALITY_GATE	substantive body 43 chars < minimum 80 chars	NO
2026-08-27 12:30:14	coupang_to_threads	NOT_PUBLISHED_NO_OPERATOR_APPROVAL	no operator approval within 10 min -> not published (fail-closed)	NO
2026-08-27 21:00:12	backlink_to_sns	BLOCKED_BY_QUALITY_GATE	substantive body 42 chars < minimum 80 chars	NO
2026-08-27 21:00:12	backlink_to_sns	BLOCKED_BY_QUALITY_GATE	substantive body 43 chars < minimum 80 chars	NO
2026-08-28 12:30:08	coupang_to_threads	META_API_REJECTED_APP_DEACTIVATED	Threads API returned OAuthException code 200 'API access deactivated'	NO
2026-08-28 21:00:10	backlink_to_sns	BLOCKED_BY_QUALITY_GATE	substantive body 42 chars < minimum 80 chars	NO
2026-08-28 21:00:10	backlink_to_sns	BLOCKED_BY_QUALITY_GATE	substantive body 43 chars < minimum 80 chars	NO

2026-08-29 12:54:02	coupang_to_threads	META_API_REJECTED_APP_DEACTIVATED	Threads API returned OAuthException code 200 'API access deactivated'	NO
2026-08-29 21:00:11	backlink_to_sns	BLOCKED_BY_QUALITY_GATE	substantive body 42 chars < minimum 80 chars	NO
2026-08-29 21:00:11	backlink_to_sns	BLOCKED_BY_QUALITY_GATE	substantive body 43 chars < minimum 80 chars	NO
2026-08-30 12:30:28	coupang_to_threads	NOT_PUBLISHED_NO_OPERATOR_APPROVAL	no operator approval within 10 min -> not published (fail-closed)	NO
2026-08-30 21:00:21	backlink_to_sns	BLOCKED_BY_QUALITY_GATE	substantive body 42 chars < minimum 80 chars	NO
2026-08-30 21:00:21	backlink_to_sns	BLOCKED_BY_QUALITY_GATE	substantive body 43 chars < minimum 80 chars	NO
2026-08-31 12:30:27	coupang_to_threads	NOT_PUBLISHED_NO_OPERATOR_APPROVAL	no operator approval within 10 min -> not published (fail-closed)	NO
2026-08-31 21:00:02	backlink_to_sns	BLOCKED_BY_QUALITY_GATE	substantive body 42 chars < minimum 80 chars	NO
2026-08-31 21:00:02	backlink_to_sns	BLOCKED_BY_QUALITY_GATE	substantive body 43 chars < minimum 80 chars	NO
2026-09-01 12:30:07	coupang_to_threads	NOT_PUBLISHED_NO_OPERATOR_APPROVAL	no operator approval within 10 min -> not published (fail-closed)	NO
2026-09-01 21:00:14	backlink_to_sns	BLOCKED_BY_QUALITY_GATE	substantive body 42 chars < minimum 80 chars	NO
2026-09-01 21:00:14	backlink_to_sns	BLOCKED_BY_QUALITY_GATE	substantive body 43 chars < minimum 80 chars	NO
2026-09-02 12:30:26	coupang_to_threads	NOT_PUBLISHED_NO_OPERATOR_APPROVAL	no operator approval within 10 min -> not published (fail-closed)	NO
2026-09-03 10:46:10	backlink_to_sns	BLOCKED_BY_QUALITY_GATE	substantive body 42 chars < minimum 80 chars	NO

2026-09-03 10:46:10	backlink_to_sns	BLOCKED_BY_QUALITY_GATE	substantive body 43 chars < minimum 80 chars	NO
2026-09-03 12:30:13	coupang_to_threads	NOT_PUBLISHED_NO_OPERATOR_APPROVAL	no operator approval within 10 min -> not published (fail- closed)	NO
2026-09-07 16:41:16	coupang_to_threads	NOT_PUBLISHED_NO_OPERATOR_APPROVAL	no operator approval within 10 min -> not published (fail- closed)	NO
2026-09-08 15:29:44	coupang_to_threads	NOT_PUBLISHED_NO_OPERATOR_APPROVAL	no operator approval within 10 min -> not published (fail- closed)	NO
2026-09-08 21:00:29	backlink_to_sns	BLOCKED_BY_QUALITY_GATE	substantive body 42 chars < minimum 80 chars	NO
2026-09-08 21:00:29	backlink_to_sns	BLOCKED_BY_QUALITY_GATE	substantive body 43 chars < minimum 80 chars	NO
2026-09-09 13:52:38	coupang_to_threads	NOT_PUBLISHED_NO_OPERATOR_APPROVAL	no operator approval within 10 min -> not published (fail- closed)	NO
2026-09-10 09:45:06	backlink_to_sns	BLOCKED_BY_QUALITY_GATE	substantive body 42 chars < minimum 80 chars	NO
2026-09-10 09:45:06	backlink_to_sns	BLOCKED_BY_QUALITY_GATE	substantive body 43 chars < minimum 80 chars	NO
2026-09-10 12:30:16	coupang_to_threads	NOT_PUBLISHED_NO_OPERATOR_APPROVAL	no operator approval within 10 min -> not published (fail- closed)	NO
2026-09-10 21:00:18	backlink_to_sns	BLOCKED_BY_QUALITY_GATE	substantive body 42 chars < minimum 80 chars	NO
2026-09-10 21:00:18	backlink_to_sns	BLOCKED_BY_QUALITY_GATE	substantive body 43 chars < minimum 80 chars	NO
2026-09-11 12:30:19	coupang_to_threads	NOT_PUBLISHED_NO_OPERATOR_APPROVAL	no operator approval within 10 min -> not published (fail- closed)	NO

2026-09-11 21:00:24	backlink_to_sns	BLOCKED_BY_QUALITY_GATE	substantive body 42 chars < minimum 80 chars	NO
2026-09-11 21:00:24	backlink_to_sns	BLOCKED_BY_QUALITY_GATE	substantive body 43 chars < minimum 80 chars	NO
2026-09-12 14:11:24	coupang_to_threads	NOT_PUBLISHED_NO_OPERATOR_APPROVAL	no operator approval within 10 min -> not published (fail- closed)	NO
2026-09-12 21:00:21	backlink_to_sns	BLOCKED_BY_QUALITY_GATE	substantive body 42 chars < minimum 80 chars	NO
2026-09-12 21:00:21	backlink_to_sns	BLOCKED_BY_QUALITY_GATE	substantive body 43 chars < minimum 80 chars	NO
2026-09-14 12:30:23	coupang_to_threads	NOT_PUBLISHED_NO_OPERATOR_APPROVAL	no operator approval within 10 min -> not published (fail- closed)	NO

Context: the week before remediation (2026-08-18 → 2026-08-25 13:29)

Shown for completeness. These runs pre-date the gate; every one failed because the app was already deactivated.

Started (KST)	Pipeline	Outcome	Detail	Published?
2026-08-24 11:30:02	newspick_to_threads	META_API_REJECTED_APP_DEACTIVATED	Threads API returned OAuthException code 200 'API access deactivated'	NO
2026-08-24 12:30:05	coupang_to_threads	META_API_REJECTED_APP_DEACTIVATED	Threads API returned OAuthException code 200 'API access deactivated'	NO
2026-08-24 13:00:23	backlink_to_sns	META_API_REJECTED_APP_DEACTIVATED	Threads API returned OAuthException code 200 'API access deactivated'	NO
2026-08-24 13:00:23	backlink_to_sns	META_API_REJECTED_APP_DEACTIVATED	Threads API returned OAuthException code 200 'API access deactivated'	NO
2026-08-24 14:00:21	newspick_to_threads	META_API_REJECTED_APP_DEACTIVATED	Threads API returned OAuthException code 200 'API access deactivated'	NO

2026-08-24 16:00:10	aliexpress_to_threads	NO_THREADS_ATTEMPT_SOURCE_ERROR	source fetch failed before any Threads call (crawl timeout / session)	NO
2026-08-24 17:30:05	coupang_to_threads	META_API_REJECTED_APP_DEACTIVATED	Threads API returned OAuthException code 200 'API access deactivated'	NO
2026-08-24 19:00:06	coupang_to_threads	META_API_REJECTED_APP_DEACTIVATED	Threads API returned OAuthException code 200 'API access deactivated'	NO
2026-08-24 20:00:23	newspick_to_threads	META_API_REJECTED_APP_DEACTIVATED	Threads API returned OAuthException code 200 'API access deactivated'	NO
2026-08-24 21:00:01	backlink_to_sns	META_API_REJECTED_APP_DEACTIVATED	Threads API returned OAuthException code 200 'API access deactivated'	NO
2026-08-24 21:00:01	backlink_to_sns	META_API_REJECTED_APP_DEACTIVATED	Threads API returned OAuthException code 200 'API access deactivated'	NO
2026-08-25 09:55:32	coupang_to_threads	META_API_REJECTED_APP_DEACTIVATED	Threads API returned OAuthException code 200 'API access deactivated'	NO
2026-08-25 09:56:50	newspick_to_threads	META_API_REJECTED_APP_DEACTIVATED	Threads API returned OAuthException code 200 'API access deactivated'	NO
2026-08-25 11:30:19	newspick_to_threads	META_API_REJECTED_APP_DEACTIVATED	Threads API returned OAuthException code 200 'API access deactivated'	NO
2026-08-25 12:30:03	coupang_to_threads	META_API_REJECTED_APP_DEACTIVATED	Threads API returned OAuthException code 200 'API access deactivated'	NO
2026-08-25 13:00:10	backlink_to_sns	META_API_REJECTED_APP_DEACTIVATED	Threads API returned OAuthException code 200 'API access deactivated'	NO
2026-08-25 13:00:10	backlink_to_sns	META_API_REJECTED_APP_DEACTIVATED	Threads API returned OAuthException code 200 'API access deactivated'	NO

6. Verification

All gate code was introduced in one commit and has not been modified since. A reviewer may request read access to the repository. Commit details:

```
3ae11a7b8bba9330f801cc94435348e43ca7088f
Author: KYUSEOK SONG
Date: Tue Aug 25 13:34:57 2026 +0900
Correct the app id in the Meta appeal doc

71b8241cf7cc72d96bde77c831d62172be5ae35c
Author: KYUSEOK SONG
Date: Tue Aug 25 13:29:29 2026 +0900
Gate Threads publishing behind quality, volume, and human approval
```

Commits touching the gate files after 71b8241: **none**.

Files in that commit: common/threads_guard.py (new, 300 lines), common/threads_approval.py (new, 321 lines), publishers/threads.py (gate wired into post/reply), pipelines/coupang_to_threads.py and pipelines/newspick_to_threads.py (fallback publishing removed), pipelines/tistory_bridge.py (approval reply routing), common/run_diagnosis.py, common/notifier.py, docs/meta-threads-appeal-2026-08.md.

Appendix A — common/threads_guard.py (quality + volume gate, full source)

""Threads 발행 게이트 - Meta 플랫폼 약관 7.e.i.2 시정 조치.

2026-08-18 Meta 가 앱(THREADS_APP_ID)의 API 접근을 차단했다. 사유는 플랫폼 약관 7.e.i.2 - "플랫폼.제품.데이터.사용자에게 부정적 영향". 포괄 조항이라 Meta 는 구체적 행위를 지목하지 않았고, 실측(473건, 2026-05-14~08-17)에서 드러난 개연성 높은 원인은 셋이었다.

1. 자동 게시 슬롯 하루 11회 (뉴스픽 4 + 쿠팡 4 + 백링크 2 + 알리 1)
2. 제휴 링크 포함 게시물이 전체의 57.3% (271/473)
3. AI 생성 실패 시 풀백 발행 - 뉴스픽은 원문 기사 제목을 그대로 내보내
한국어 제정에 베트남어/영어 게시물이 섞였고, 쿠팡은 상품명만 덜렁 나갔다

이 모듈은 2.3을 코드에 내장된 게이트로 막는다. 1은 .env 스케줄로 처리한다.

****게이트를 `publishers/threads.py` 진입점에 두는 이유****: 파이프라인마다 넣으면 새 파이프라인이 누락시킬 수 있다. Publisher 진입점은 모든 Threads 발행이 반드시 통과하는 유일한 지점이라, 설정값을 되돌리는 것만으로는 우회되지 않는다 (Meta 소명의 "코드에 내장" 근거).

환경변수:

THREADS_MAX_PER_DAY	하루 총 게시물 상한 (기본 3)
THREADS_MAX_AFFILIATE_PER_DAY	제휴 게시물 하루 상한 (기본 1)
THREADS_MAX_AFFILIATE_RATIO	최근 30일 제휴 비중 상한 (기본 0.30)
THREADS_MIN_CHARS	실질 본문 최소 길이 (기본 80)
THREADS_REQUIRE_KOREAN	한국어 이외 차단 (기본 true)
THREADS_MIN_KOREAN_RATIO	한글 최소 비율 (기본 0.30)
THREADS_GUARD_ENABLED	게이트 전체 on/off (기본 true)

CLI:

```
python -m common.threads_guard # 오늘 소진 현황
"""
from __future__ import annotations

import json
import os
import re
import sys
from datetime import date, datetime, timedelta
from pathlib import Path

from common.logger import log


_STATE_PATH = Path(__file__).resolve().parent.parent / "data" / "threads_guard.json"

# 비중 계산 창 - 하루치만 보면 "오늘 1건이 제휴" 가 곧 100% 라 상한이 무의미해진다.
_RATIO_WINDOW_DAYS = 30

# 실질 본문 길이/언어 판정에서 제외할 요소.
#   - 의무 고지문: 전 제휴 게시물에 동일하게 붙여 길이·한글비율을 인위적으로 올린다
#   - URL / 해시태그 / 이모지: 본문 실질이 아니다
_NOTICE_RE = re.compile(
    r"\[?광고\]?\s*이\s*게시물은.*?활동의\s*일환으로.*?제공받습니다\.?\"", re.S)
_URL_RE = re.compile(r"https?:\/\/\S+")
_TAG_RE = re.compile(r"#\S+")
_NON_TEXT_RE = re.compile(
    r"[\U0001F000-\U0001FAFF*- \t←→\s]+")
_HANGUL_RE = re.compile(r"[가-힣]")

# -----
# 설정
# -----
```

```

def _int_env(name: str, default: int) -> int:
    try:
        return int(str(os.getenv(name, "")).strip() or default)
    except ValueError:
        return default

def _float_env(name: str, default: float) -> float:
    try:
        return float(str(os.getenv(name, "")).strip() or default)
    except ValueError:
        return default

def _bool_env(name: str, default: bool = True) -> bool:
    raw = str(os.getenv(name, "")).strip().lower()
    if not raw:
        return default
    return raw not in ("0", "false", "no", "off")

def enabled() -> bool:
    return _bool_env("THREADS_GUARD_ENABLED", True)

# -----
# 상태 파일
# -----

def _load() -> dict:
    if not _STATE_PATH.exists():
        return {}
    try:
        data = json.loads(_STATE_PATH.read_text(encoding="utf-8"))
        return data if isinstance(data, dict) else {}
    except Exception:
        # 손상 시 빈 상태로 시작 - 카운터가 리셋되면 그날 상한이 느슨해질 뿐이고,
        # 여기서 예외를 올리면 발행 경로 전체가 죽는다.
        return {}

def _save(data: dict) -> None:
    try:
        _STATE_PATH.parent.mkdir(parents=True, exist_ok=True)
        _STATE_PATH.write_text(json.dumps(data, ensure_ascii=False, indent=2),
                                encoding="utf-8")
    except Exception as e:
        log(f"[threads_guard] 상태 저장 실패 (무시): {e}", "warn")

def _today() -> str:
    return date.today().isoformat()

def _window_totals(data: dict, days: int = _RATIO_WINDOW_DAYS) -> tuple[int, int]:
    """최근 days 일의 (총 게시물, 제휴 게시물) 합계."""
    cutoff = date.today() - timedelta(days=days)
    total = affiliate = 0
    for day, rec in data.items():
        try:
            if datetime.strptime(day, "%Y-%m-%d").date() < cutoff:
                continue
        except ValueError:
            continue
        total += int(rec.get("total", 0) or 0)
        affiliate += int(rec.get("affiliate", 0) or 0)
    return total, affiliate

def today_usage() -> dict:
    """오늘 소진 현황 - 진단/CLI 용."""

```

```

data = _load()
rec = data.get(_today(), {})
win_total, win_aff = _window_totals(data)
return {
    "date": _today(),
    "total": int(rec.get("total", 0) or 0),
    "affiliate": int(rec.get("affiliate", 0) or 0),
    "max_total": _int_env("THREADS_MAX_PER_DAY", 3),
    "max_affiliate": _int_env("THREADS_MAX_AFFILIATE_PER_DAY", 1),
    "window_total": win_total,
    "window_affiliate": win_aff,
    "window_ratio": (win_aff / win_total) if win_total else 0.0,
    "max_ratio": _float_env("THREADS_MAX_AFFILIATE_RATIO", 0.30),
}

```

```

#
# 품질 검사
#

```

```

def _substantive_text(text: str) -> str:
    """의무 고지문·URL·해시태그·이모지를 건너낸 실질 본문."""
    t = _NOTICE_RE.sub(" ", text or "")
    t = _URL_RE.sub(" ", t)
    t = _TAG_RE.sub(" ", t)
    return _NON_TEXT_RE.sub(" ", t).strip()

```

```

def check_quality(text: str) -> tuple[bool, str]:
    """본문 품질 검사. (통과여부, 사유) 반환.

```

AI 생성이 실패했을 때 파이프라인이 상품명·원문 기사 제목만으로 폴백 발행하던 경로를 여기서 끊는다. 실측상 이 폴백이 저품질 게시물과 언어 불일치 게시물 (한국어 계정에 베트남어/영어 원문) 양쪽의 실제 원인이었다.

```

    """
    body = _substantive_text(text)

    min_chars = _int_env("THREADS_MIN_CHARS", 80)
    if len(body) < min_chars:
        return False, f"실질 본문 {len(body)}자 < 최소 {min_chars}자 (AI 생성 실패 폴백 의심)"

    if _bool_env("THREADS_REQUIRE_KOREAN", True):
        letters = [c for c in body if c.isalpha()]
        if letters:
            ratio = len(_HANGUL_RE.findall(body)) / len(letters)
            min_ratio = _float_env("THREADS_MIN_KOREAN_RATIO", 0.30)
            if ratio < min_ratio:
                return False, (f"한글 비율 {ratio:.0%} < {min_ratio:.0%} - "
                               f"계정 주 언어(한국어)와 불일치")

    return True, ""

```

```

#
# 발행량 검사
#

```

```

def check_quota(is_affiliate: bool) -> tuple[bool, str]:
    """일일 상한·제휴 비중 검사. (통과여부, 사유) 반환."""
    data = _load()
    rec = data.get(_today(), {})
    total = int(rec.get("total", 0) or 0)
    aff = int(rec.get("affiliate", 0) or 0)

    max_total = _int_env("THREADS_MAX_PER_DAY", 3)
    if total >= max_total:
        return False, f"오늘 게시물 {total}건 - 일일 상한 {max_total}건 도달"

    if is_affiliate:
        max_aff = _int_env("THREADS_MAX_AFFILIATE_PER_DAY", 1)
        if aff >= max_aff:

```

```

        return False, f"오늘 제휴 게시물 {aff}건 - 일일 상한 {max_aff}건 도달"

    win_total, win_aff = _window_totals(data)
    max_ratio = _float_env("THREADS_MAX_AFFILIATE_RATIO", 0.30)
    # 표본이 적을 때(창 내 10건 미만)는 비중이 요동쳐 정상 발행까지 막는다.
    # 일일 상한이 이미 걸려 있으므로 비중은 표본이 쌓인 뒤에만 적용한다.
    if win_total >= 10:
        projected = (win_aff + 1) / (win_total + 1)
        if projected > max_ratio:
            return False, (f"제휴 비중 {projected:.0%} > 상한 {max_ratio:.0%} "
                           f"(최근 {_RATIO_WINDOW_DAYS}일 {win_aff}/{win_total})")

    return True, ""

def record_published(is_affiliate: bool) -> None:
    """발행 성공을 카운터에 반영."""
    data = _load()
    rec = data.setdefault(_today(), {"total": 0, "affiliate": 0})
    rec["total"] = int(rec.get("total", 0) or 0) + 1
    if is_affiliate:
        rec["affiliate"] = int(rec.get("affiliate", 0) or 0) + 1

    # 창 밖 기록은 비중 계산에 쓰이지 않으므로 정리한다 (파일 무한 증가 방지).
    cutoff = date.today() - timedelta(days=_RATIO_WINDOW_DAYS * 2)
    for day in list(data.keys()):
        try:
            if datetime.strptime(day, "%Y-%m-%d").date() < cutoff:
                del data[day]
        except ValueError:
            del data[day]
    _save(data)

# -----
# 통합 진입점
# -----

def is_affiliate_text(text: str) -> bool:
    """본문이 상업(제휴) 게시물인지 판정.

    고지문 존재 여부로 보는 게 가장 확실하다 - 제휴 링크가 들어가는 모든 경로가
    common/affiliate_notice.py 를 통과하기 때문이다. 단축 URL 이 섞여 링크 도메인
    으로는 판정할 수 없는 케이스도 고지문으로는 잡힌다.
    """
    if not text:
        return False
    low = text.lower()
    if "affiliate-notice" in low or "파트너스 활동의 일환" in text:
        return True
    return any(d in low for d in (
        "coupang.com", "coupa.ng", "link.coupang",
        "aliexpress.com", "s.click.aliexpress",
    ))

def precheck(text: str) -> tuple[bool, str, bool]:
    """발행 직전 통합 검사.

    Returns:
        (통과여부, 차단사유, 제휴여부)
    """
    affiliate = is_affiliate_text(text)
    if not enabled():
        return True, "", affiliate

    ok, reason = check_quality(text)
    if not ok:
        return False, reason, affiliate

    ok, reason = check_quota(affiliate)

```

```

    if not ok:
        return False, reason, affiliate

    return True, "", affiliate

def main() -> None:
    try:
        sys.stdout.reconfigure(encoding="utf-8") # type: ignore[attr-defined]
    except Exception:
        pass
    u = today_usage()
    print("=== Threads 발행 게이트 현황 ===")
    print(f"    활성화      : {enabled()}")
    print(f"    오늘({u['date']})")
    print(f"    총 게시물   : {u['total']} / {u['max_total']}")
    print(f"    제휴       : {u['affiliate']} / {u['max_affiliate']}")
    print(f"    최근 {_RATIO_WINDOW_DAYS}일")
    print(f"    총 게시물   : {u['window_total']}")
    print(f"    제휴 비중   : {u['window_ratio']:.1%} (상한 {u['max_ratio']:.0%})")

if __name__ == "__main__":
    main()

```

Appendix B — common/threads_approval.py (human approval gate, full source)

"""Threads 발행 사전 승인 - Meta 플랫폼 약관 7.e.i.2 시정 조치.

무인 자동 게시를 중단하고, 모든 Threads 게시물이 발행 전에 텔레그램으로 운영자에게 전달되어 **승인한 건만** 게시되도록 한다.

구조 (프로세스 간 통신이 필요한 이유):

파이프라인은 scheduler_runner 가 띄우는 **별도 subprocess** 이고, 텔레그램 응답을 받는 long-poll 스레드는 **스케줄러 프로세스** 안에 있다. (pipelines/tistory_bridge.py 의 _telegram_long_poll_loop - DKAPTCHA 답변 수신용으로 이미 상시 동작 중)

따라서 캡처처럼 in-memory dict 로는 오갈 수 없고, 파일을 매개로 한다:

파이프라인 subprocess	스케줄러 프로세스 (bridge)
request() └─ 텔레그램 전송(force_reply) └─ data/threads_approvals.json └─ pending 기록 wait() - 파일 폴링 └─ approved → 발행 / 그 외 → 미발행	_telegram_long_poll_loop └─ 답글 수신 └─ resolve_by_tg_message_id()

getUpdates 는 offset 으로 확정되면 다른 소비자가 같은 업데이트를 볼 수 없다. 그래서 파이프라인이 별도로 getUpdates 를 돌리면 브릿지의 캡처 수신을 훔쳐간다 - 브릿지가 살아있는 동안에는 절대 직접 폴링하지 않고 파일만 본다. 브릿지가 없을 때(수동 실행 등)만 자체 폴링으로 풀백한다.

승인 규칙:

답글이 승인 키워드면 발행, 그 외 텍스트는 거부로 본다. 무응답은 발행하지 않는다(fail-closed) - 승인 없이 게시되는 경로를 남기지 않기 위함이다.

환경변수:

THREADS_APPROVAL_REQUIRED 승인 게이트 on/off (기본 true)
THREADS_APPROVAL_TIMEOUT 승인 대기 초 (기본 600)
TELEGRAM_BOT_TOKEN / TELEGRAM_CHAT_ID

"""

from __future__ import annotations

import json
import os
import re
import time
import uuid
from datetime import datetime, timedelta
from pathlib import Path
from typing import Optional

from common.logger import log

_PATH = Path(__file__).resolve().parent.parent / "data" / "threads_approvals.json"

승인 키워드에 정확히 일치할 때만 발행한다. 나머지는 전부 거부로 본다 -
"잠깐만", "이건 좀" 같은 답글을 승인으로 오독하는 경로를 남기지 않기 위함.
_APPROVE_WORDS = {"ok", "o", "y", "yes", "예", "승인", "발행", "확인", "o", "👍"}

브릿지 생존 확인 - 살아있으면 절대 자체 getUpdates 를 돌리지 않는다.

_BRIDGE_HEALTH = "http://127.0.0.1:5757/healthz"

def required() -> bool:
 raw = str(os.getenv("THREADS_APPROVAL_REQUIRED", "")).strip().lower()
 if not raw:

```

        return True
    return raw not in ("0", "false", "no", "off")

def _timeout_sec() -> int:
    try:
        return int(str(os.getenv("THREADS_APPROVAL_TIMEOUT", "")).strip() or 600)
    except ValueError:
        return 600

def _fmt_wait(sec: int) -> str:
    """대기 시간 표기 - 60초 미만을 '0분' 으로 쓰지 않기 위함."""
    return f"{sec // 60}분" if sec >= 60 else f"{sec}초"

# -----
# 상태 파일 (atomic write - 두 프로세스가 같은 파일을 만진다)
# -----

def _load() -> dict:
    if not _PATH.exists():
        return {}
    try:
        data = json.loads(_PATH.read_text(encoding="utf-8"))
        return data if isinstance(data, dict) else {}
    except Exception:
        return {}

def _save(data: dict) -> None:
    _PATH.parent.mkdir(parents=True, exist_ok=True)
    tmp = _PATH.with_suffix(".tmp")
    tmp.write_text(json.dumps(data, ensure_ascii=False, indent=2), encoding="utf-8")
    tmp.replace(_PATH)

def _prune(data: dict) -> dict:
    """24시간 지난 기록 제거 - 파일 무한 증가 방지."""
    cutoff = datetime.now() - timedelta(hours=24)
    for aid in list(data.keys()):
        try:
            if datetime.fromisoformat(data[aid].get("created_at", "")) < cutoff:
                del data[aid]
        except Exception:
            del data[aid]
    return data

# -----
# 텔레그램
# -----

def _send_request_message(text: str, meta: str) -> Optional[int]:
    """승인 요청 메시지 발송. message_id 반환 (실패 시 None)."""
    try:
        import requests
    except ImportError:
        log("[threads_approval] requests 미설치 - 승인 요청 불가", "error")
        return None

    token = os.getenv("TELEGRAM_BOT_TOKEN", "").strip()
    chat_id = os.getenv("TELEGRAM_CHAT_ID", "").strip()
    if not token or not chat_id:
        log("[threads_approval] TELEGRAM_BOT_TOKEN/CHAT_ID 미설정 - 승인 요청 불가", "error")
        return None

    body = (
        f"👤 Threads 발행 승인 요청\n"
        f"-----\n"

```

```

f"{meta}\n"
f"_____ \n"
f"{text}\n"
f"_____ \n"
f"이 메시지에 답글로 승인/거부하세요.\n"
f"  승인: ok / 승인 / o\n"
f"  거부: no / 거부 / x\n"
f"※ {_fmt_wait(_timeout_sec())} 내 무응답 시 발행하지 않습니다."
)
try:
    r = requests.post(
        f"https://api.telegram.org/bot{token}/sendMessage",
        json={
            "chat_id": chat_id,
            "text": body[:4096],
            "disable_web_page_preview": True,
            "reply_markup": {
                "force_reply": True,
                "input_field_placeholder": "ok / no",
            },
        },
        timeout=15,
    )
    if not r.ok:
        log(f"[threads_approval] sendMessage {r.status_code}: {r.text[:200]}",
            "error")
        return None
    return r.json().get("result", {}).get("message_id")
except Exception as e:
    log(f"[threads_approval] sendMessage 예외: {e}", "error")
    return None

def _bridge_alive() -> bool:
    try:
        import requests
        return requests.get(_BRIDGE_HEALTH, timeout=2).ok
    except Exception:
        return False

def _decide(reply_text: str) -> str:
    """답글 텍스트 → approved / rejected."""
    t = (reply_text or "").strip().lower()
    t = re.sub(r"[\s]+$", "", t)
    if t in _APPROVE_WORDS:
        return "approved"
    return "rejected"

# _____
# 파이프라인 측 API
# _____

def request(text: str, meta: str = "") -> Optional[str]:
    """승인 요청 등록 + 텔레그램 발송. approval_id 반환 (실패 시 None)."""
    msg_id = _send_request_message(text, meta)
    if not msg_id:
        return None

    aid = uuid.uuid4().hex[:12]
    data = _prune(_load())
    data[aid] = {
        "tg_message_id": msg_id,
        "status": "pending",
        "meta": meta,
        "preview": text[:200],
        "created_at": datetime.now().isoformat(timespec="seconds"),
        "decided_at": "",
        "reply": "",
    }
}

```

```

_save(data)
log(f"[threads_approval] 승인 요청 발송 (id={aid}, tg_msg={msg_id})", "step")
return aid

def wait(approval_id: str, timeout_sec: Optional[int] = None) -> tuple[bool, str]:
    """승인 결과 대기. (승인여부, 사유) 반환.

    브릿지가 살아있으면 파일만 폴링한다. 브릿지가 없으면(수동 실행 등) 자체
    getUpdates 로 폴백한다 - 이때는 홈쳐갈 상대가 없으므로 충돌하지 않는다.
    """
    timeout_sec = timeout_sec or _timeout_sec()
    deadline = time.time() + timeout_sec
    use_self_poll = not _bridge_alive()
    if use_self_poll:
        log("[threads_approval] 브릿지 미가동 - 자체 폴링으로 승인 대기", "info")

    offset = 0
    while time.time() < deadline:
        data = _load()
        rec = data.get(approval_id)
        if not rec:
            return False, "승인 요청 기록이 사라짐"
        if rec.get("status") == "approved":
            return True, ""
        if rec.get("status") == "rejected":
            return False, f"운영자 거부 (답글: {rec.get('reply', '')[:30]})"

        if use_self_poll:
            offset = _self_poll_once(offset)

        time.sleep(3)

    _mark(approval_id, "timeout", "")
    return False, f"[_fmt_wait(timeout_sec)] 내 승인 없음 - 발행하지 않음"

def _self_poll_once(offset: int) -> int:
    """브릿지가 없을 때만 쓰는 getUpdates 1회. 새 offset 반환."""
    try:
        import requests
        token = os.getenv("TELEGRAM_BOT_TOKEN", "").strip()
        if not token:
            return offset
        r = requests.get(f"https://api.telegram.org/bot{token}/getUpdates",
                        params={"offset": offset, "timeout": 0}, timeout=10)
        if not r.ok:
            return offset
        for upd in r.json().get("result", []):
            offset = upd["update_id"] + 1
            msg = upd.get("message", {})
            reply_to = msg.get("reply_to_message")
            if not reply_to:
                continue
            resolve_by_tg_message_id(reply_to.get("message_id"),
                                    (msg.get("text") or "").strip())
    except Exception:
        pass
    return offset

# -----
# 브릿지 측 API
# -----

def find_by_tg_message_id(msg_id: int) -> Optional[str]:
    """텔레그램 message_id 로 pending 승인 요청 찾기."""
    if not msg_id:
        return None
    for aid, rec in _load().items():
        if rec.get("tg_message_id") == msg_id and rec.get("status") == "pending":

```

```

        return aid
    return None

def resolve_by_tg_message_id(msg_id: int, reply_text: str) -> bool:
    """답글 수신 → 승인/거부 확정. 처리했으면 True.

    브릿지의 텔레그램 long-poll 이 호출한다. 캡차 답변과 같은 채널을 쓰므로,
    매칭되는 pending 승인 요청이 없으면 False 를 반환해 호출자가 캡차 처리로
    넘어가게 한다.
    """
    aid = find_by_tg_message_id(msg_id)
    if not aid:
        return False
    status = _decide(reply_text)
    _mark(aid, status, reply_text)
    log(f"[threads_approval] {status} (id={aid}, 답글={reply_text[:20]})", "ok")
    return True

def _mark(approval_id: str, status: str, reply_text: str) -> None:
    data = _load()
    rec = data.get(approval_id)
    if not rec:
        return
    rec["status"] = status
    rec["reply"] = reply_text
    rec["decided_at"] = datetime.now().isoformat(timespec="seconds")
    _save(data)

# -----
# 통합 진입점
# -----

def gate(text: str, meta: str = "") -> tuple[bool, str]:
    """발행 직전 승인 게이트. (통과여부, 사유) 반환."""
    if not required():
        return True, ""
    aid = request(text, meta)
    if not aid:
        # 승인 요청을 보내지 못하면 발행하지 않는다. 여기서 통과시키면
        # '텔레그램이 죽으면 무인 발행으로 되돌아가는' 우회로가 생긴다.
        return False, "승인 요청 발송 실패 - 발행하지 않음"
    return wait(aid)

```

Appendix C — git diff of commit 71b8241: publisher entry point and pipeline fallback removal

commit 71b8241cf7cc72d96bde77c831d62172be5ae35c
Author: KYUSEOK SONG <seokcess@glitzy.kr>
Date: Tue Aug 25 13:29:29 2026 +0900

Gate Threads publishing behind quality, volume, and human approval

Meta deactivated the app's API access on 2026-08-18 under Platform Terms 7.e.i.2 (negatively affecting the platform/products/data/users). That code 200 "API access deactivated" is app-level, not token-level: calls using only app_id+secret fail too, so re-issuing the token does nothing.

Analysis of 473 posts (05-14..08-17) surfaced three likely causes: 11 automated slots/day, 57% affiliate-link posts, and a fallback that published placeholder text when AI generation failed -- that fallback shipped raw article titles in Vietnamese/English to a Korean account, and bodies that were a single product name.

The gates live at the ThreadsPublisher entry point rather than in each pipeline, since that is the one path every Threads post must take; a new pipeline cannot forget to call them, and reverting a config value does not bypass them.

Approval crosses a process boundary: pipelines run as scheduler subprocesses while the Telegram long-poll runs inside the scheduler, so requests are handed over via data/threads_approvals.json instead of memory. Pipelines must not poll getUpdates while the bridge is alive -- offset confirmation would steal the bridge's DKAPTCHA replies -- so self-polling is a fallback for when no bridge is running.

Also maps the app-deactivated and token-expired responses to distinct diagnosis labels; for a week they surfaced only as "container creation failed", which is why the outage went unexplained.

Co-Authored-By: Claude Opus 5 (1M context) <noreply@anthropic.com>

```
diff --git a/pipelines/coupang_to_threads.py b/pipelines/coupang_to_threads.py
index 715e6a5..46c694d 100644
--- a/pipelines/coupang_to_threads.py
+++ b/pipelines/coupang_to_threads.py
@@ -17,8 +17,8 @@
     타임라인에서 독립 게시물로 노출되므로).
```

실행:

```
- python -m pipelines.coupang_to_threads # chain (기본)
- python -m pipelines.coupang_to_threads --mode single
+ python -m pipelines.coupang_to_threads # single (기본)
+ python -m pipelines.coupang_to_threads --mode chain
  python -m pipelines.coupang_to_threads --keyword 무선이어폰
"""
from __future__ import annotations
@@ -37,6 +37,7 @@ from common.ai_intro import (
)
from common.logger import log
from common.url_shortener import shorten as shorten_url
+from publishers.base import PostResult
from publishers.threads import ThreadsPublisher
from sources.coupang import CoupangSource

@@ -62,7 +63,7 @@ def _shorten(url: str) -> str:

def _publish_single(pub: ThreadsPublisher, kw: str, product: dict,
- short_link: str) -> "object":
```

```

+         short_link: str) -> PostResult:
+         """단일 게시물 발행 (기본 모드).

한 게시물에 후킹 + 디테일 + 마무리 모두 담기므로 chain 보다 글자 수
@@ -70,7 +71,12 @@ def _publish_single(pub: ThreadsPublisher, kw: str, product: dict,
+         """
+         caption = generate_threads_caption(kw, product, short_url=short_link, max_chars=230)
+         if not caption:
+         -         caption = (product.get("name", "") or "")[:60]
+         +         # 예전에는 상품명만 잘라 풀백 발행했다. 그렇게 나간 게시물은 본문이
+         +         # 사실상 상품명 한 줄뿐이라 저품질 신호가 되고, Meta 플랫폼 약관
+         +         # 7.e.i.2 조치(2026-08-18)의 개연성 높은 원인 중 하나였다.
+         +         # 생성 실패는 발행 취소로 처리한다.
+         +         log("AI 캡션 생성 실패 - 발행 취소 (상품명 풀백 금지)", "error")
+         +         return PostResult(success=False, message="AI 캡션 생성 실패")

body = caption
if short_link:
@@ -86,7 +92,7 @@ def _publish_single(pub: ThreadsPublisher, kw: str, product: dict,

def _publish_chain(pub: ThreadsPublisher, kw: str, product: dict,
-         short_link: str) -> "object":
+         short_link: str) -> PostResult:
+         """3편 reply chain 발행 - Threads 알고리즘 우회 패턴."""
+         chain_parts = generate_threads_chain(
+             kw, product, short_url=short_link, max_chars_each=150)
diff --git a/pipelines/newspick_to_threads.py b/pipelines/newspick_to_threads.py
index 43c4a54..5c7638a 100644
--- a/pipelines/newspick_to_threads.py
+++ b/pipelines/newspick_to_threads.py
@@ -61,10 +61,17 @@ def run(category: str = "추천") -> None:
+         log(f"기사: {title[:50]}", "info")

# 2) AI Threads 캡션
+         #
+         # 생성 실패 시 예전에는 "제목 + 📌 자세한 내용은 아래에서" 로 풀백 발행했는데,
+         # 뉴스픽 기사 제목은 원문 언어 그대로라 한국어 계정에 베트남어/영어 게시물이
+         # 섞여 나갔다. 이것이 Meta 플랫폼 약관 7.e.i.2 조치(2026-08-18)의 개연성
+         # 높은 원인 중 하나여서, 풀백을 없애고 생성 실패는 발행 취소로 처리한다.
+         caption = generate_newspick_threads_caption(title, category=category, max_chars=230)
+         if not caption:
+         -         # 풀백 - 단순 제목 + 안내
+         -         caption = f"{title}\n\n📌 자세한 내용은 아래에서"
+         +         log("AI 캡션 생성 실패 - 발행 취소 (원문 제목 풀백 금지)", "error")
+         +         from common.notifier import notify_pipeline_result
+         +         notify_pipeline_result("뉴스픽→Threads", 0, 1, details="AI 캡션 생성 실패")
+         +         return

# 3) 본문 = 캡션 + 빈 줄 + 단축링크 (뉴스픽 고지는 미부착)
body = caption
diff --git a/pipelines/tistory_bridge.py b/pipelines/tistory_bridge.py
index a5860cb..1056ebc 100644
--- a/pipelines/tistory_bridge.py
+++ b/pipelines/tistory_bridge.py
@@ -442,10 +442,20 @@ def _telegram_send_photo(image_b64: str, caption: str) -> Optional[int]:

def _telegram_long_poll_loop() -> None:
-         """텔레그램 봇 getUpdates long-poll - 사용자가 캡차 메시지에 답글 달면 수신.
+         """텔레그램 봇 getUpdates long-poll - 사용자 답글을 수신해 라우팅.

-         reply_to_message.message_id 로 어느 캡차 요청의 답변인지 매칭 →
-         set_captcha_answer() 로 저장. content.js 가 polling 으로 가져감.
+         reply_to_message.message_id 로 무엇에 대한 답글인지 매칭한다:
+
+         1. Threads 발행 승인 (common/threads_approval.py) - 먼저 확인
+         2. DKAPTCHA 답안 (common/tistory_queue.py)
+
+         Threads 승인을 먼저 보는 이유: 승인 요청은 파일(data/threads_approvals.json)
+         에 pending 으로 남아 있어 매칭이 확정적이지만, 캡차는 in-memory 라 이미

```

```

+ 처리된 뒤일 수 있다. 매칭 실패 시 False 를 받아 캡차 경로로 넘어간다.
+
+ getUpdates 는 offset 으로 확정되면 다른 소비자가 같은 업데이트를 볼 수 없다.
+ 그래서 이 루프가 텔레그램 답글의 **유일한** 소비자여야 한다 - 파이프라인
+ subprocess 는 브릿지가 살아있으면 직접 폴링하지 않고 파일만 본다.
+ """
+     try:
+         import requests
@@ -477,6 +487,15 @@ def _telegram_long_poll_loop() -> None:
+         text = (msg.get("text") or "").strip()
+         if not text or not reply_msg_id:
+             continue
+         # 1) Threads 발행 승인 - 파일 기반이라 매칭이 확정적
+         try:
+             from common.threads_approval import resolve_by_tg_message_id
+             if resolve_by_tg_message_id(reply_msg_id, text):
+                 continue
+         except Exception as e:
+             log(f"[bridge] Threads 승인 처리 예외 (캡차로 계속): {e}", "warn")
+
+         # 2) DKAPTCHA 답안
+         item_id = find_item_by_tg_message_id(reply_msg_id)
+         if item_id:
+             set_captcha_answer(item_id, text)
diff --git a/publishers/threads.py b/publishers/threads.py
index 3f89a9a..25dea19 100644
--- a/publishers/threads.py
+++ b/publishers/threads.py
@@ -72,6 +72,12 @@ class ThreadsPublisher(Publisher):
+
+         log(f"Threads 발행 준비: {text[:60]}...", "step")
+
+         # Step 0: 발행 게이트 (품질.발행량.사람 승인)
+         meta = self._meta_line(kwargs, category=category, image_url=image_url)
+         ok, reason, is_affiliate = self._gate(text, meta)
+         if not ok:
+             return PostResult(success=False, message=reason)
+
+         # 미디어 타입
+         media_type = "IMAGE" if image_url else "TEXT"
@@ -84,7 +90,10 @@ class ThreadsPublisher(Publisher):
+         time.sleep(2)
+
+         # Step 3: 발행
+         - return self._publish_container(container_id)
+         + result = self._publish_container(container_id)
+         + if result.success:
+         +     self._record(is_affiliate)
+         + return result
+
+     def upload_image(self, local_path: str) -> str:
+         """로컬 이미지 업로드 미지원 - 공개 URL 직접 전달 방식 사용."""
@@ -108,12 +117,118 @@ class ThreadsPublisher(Publisher):
+         if len(text) > 500:
+             text = text[:497] + "..."
+         log(f"Threads reply 준비 (parent={parent_post_id[:12]}...): {text[:50]}", "step")
+
+         +
+         + # reply 도 타임라인에 독립 게시물로 노출되므로 원글과 동일하게 게이트를
+         + # 통과시킨다. chain 모드에서 1건이 3게시물로 불어나던 경로를 막는 지점.
+         + ok, reason, is_affiliate = self._gate(
+         +     text, f"reply → parent {parent_post_id[:12]}")
+         + if not ok:
+         +     return PostResult(success=False, message=reason)
+         +
+         + container_id = self._create_container(
+         +     text, media_type="TEXT", reply_to_id=parent_post_id)
+         + if not container_id:
+         +     return PostResult(success=False, message="reply 컨테이너 생성 실패")
+         + time.sleep(2)
+         - return self._publish_container(container_id)

```

```

+         result = self._publish_container(container_id)
+         if result.success:
+             self._record(is_affiliate)
+         return result
+
+ # -----
+ # 발행 게이트 (Meta 플랫폼 약관 7.e.i.2 시정 조치)
+ # -----
+
+ @staticmethod
+ def _meta_line(kwargs: dict, category: str = "", image_url: str = "") -> str:
+     """승인 메시지 상단에 붙일 한 줄 요약 - 운영자가 맥락을 보고 판단하도록."""
+     bits = []
+     for key in ("source", "keyword", "pipeline"):
+         val = kwargs.get(key)
+         if val:
+             bits.append(f"{key}={val}")
+     if category:
+         bits.append(f"category={category}")
+     if image_url:
+         bits.append("image=Y")
+     return " / ".join(bits) if bits else "(메타 정보 없음)"
+
+ def _gate(self, text: str, meta: str = "") -> tuple[bool, str, bool]:
+     """발행 직전 게이트. (통과여부, 차단사유, 제휴여부) 반환.
+
+     순서가 중요하다 - 품질·발행량을 먼저 보고 사람 승인을 마지막에 묻는다.
+     기계가 걸러낼 수 있는 것을 운영자에게 들이밀면 승인 피로가 쌓이고,
+     결국 게이트를 꺼버리게 된다.
+
+     Publisher 진입점에 두는 이유는 이곳이 모든 Threads 발행이 반드시
+     통과하는 유일한 지점이기 때문이다. 파이프라인마다 넣으면 새 파이프라인이
+     누락시킬 수 있다.
+     """
+     try:
+         from common.threads_guard import precheck
+         from common.threads_approval import gate as approval_gate
+     except Exception as e:
+         # 게이트를 불러오지 못하면 발행하지 않는다. 여기서 통과시키면
+         # import 하나가 깨지는 것만으로 무인 발행으로 되돌아간다.
+         log(f"Threads 게이트 로드 실패 - 발행 중단: {e}", "error")
+         return False, "발행 게이트 로드 실패", False
+
+     ok, reason, is_affiliate = precheck(text)
+     if not ok:
+         log(f"Threads 발행 차단 (게이트): {reason}", "error")
+         return False, f"발행 게이트 차단: {reason}", is_affiliate
+
+     ok, reason = approval_gate(text, meta)
+     if not ok:
+         log(f"Threads 발행 보류 (승인): {reason}", "error")
+         return False, f"발행 승인 없음: {reason}", is_affiliate
+
+     return True, "", is_affiliate
+
+ @staticmethod
+ def _record(is_affiliate: bool) -> None:
+     """발행 성공을 일일 카운터에 반영 (실패해도 발행을 되돌리지 않는다)."""
+     try:
+         from common.threads_guard import record_published
+         record_published(is_affiliate)
+     except Exception as e:
+         log(f"Threads 발행량 기록 실패 (무시): {e}", "warn")
+
+ def _alert_fatal_api_error(self, body: str) -> None:
+     """API 오류 본문에서 '사람이 개입해야만 풀리는' 상태만 골라 텔레그램 알림.
+
+     발행 실패의 대부분(rate-limit, 이미지 URL 거부, 본문 길이)은 다음 슬롯에
+     자동 회복되므로 전부 알리면 소음이 된다. 반대로 아래 두 상태는 개입 전까지
+     모든 슬롯이 100% 실패하는데, 그동안 stderr 에는 '컨테이너 생성 실패' 만
+     남아 원인이 드러나지 않았다 (2026-08-18~08-25, 70건 연속 무성 실패).

```

```

+
+     notify_login_required 가 24시간 throttle 을 이미 갖고 있어 슬롯마다
+     중복 발송되지 않는다.
+     """
+     low = (body or "").lower()
+     if "api access deactivated" in low:
+         platform = "Threads (Meta 앱 API 접근 차단)"
+         reason = "Meta 가 앱의 API 접근을 비활성화했습니다. 토큰 문제가 아닙니다."
+         app_id = os.getenv("THREADS_APP_ID", "").strip()
+         target = (f"https://developers.facebook.com/apps/{app_id}" if app_id
+                  else "https://developers.facebook.com/apps (앱 목록)")
+         instructions = (
+             f"{target} 에서 앱 상태.정책 경고 확인 "
+             "(토큰 재발급으로는 복구되지 않음)"
+         )
+     elif "session has expired" in low or "invalid oauth access token" in low:
+         platform = "Threads (액세스 토큰 만료)"
+         reason = "장기 액세스 토큰이 만료됐습니다."
+         instructions = "python -m common.threads_token oauth 로 재발급"
+     else:
+         return
+
+     try:
+         from common.notifier import notify_login_required
+         notify_login_required(platform, instructions, reason=reason)
+     except Exception:
+         # 알림 실패가 발행 경로를 막아선 안 된다 (notifier 는 fire-and-forget).
+         pass
+
+     def _create_container(self, text: str, media_type: str,
+                          image_url: str = "",
+                          @@ -141,6 +256,7 @@ class ThreadsPublisher(Publisher):
+         log(f"Threads 컨테이너 생성 완료: {container_id}", "ok")
+         return container_id
+         log(f"Threads 컨테이너 생성 실패 ({resp.status_code}): {resp.text[:200]}", "error")
+         self._alert_fatal_api_error(resp.text)
+         return None
+     except Exception as e:
+         log(f"Threads 컨테이너 생성 예외: {e}", "error")
+     @@ -164,6 +280,7 @@ class ThreadsPublisher(Publisher):
+         resp = requests.post(url, params=params, timeout=15)
+         if not resp.ok:
+             log(f"Threads 발행 실패 ({resp.status_code}): {resp.text[:200]}", "error")
+             self._alert_fatal_api_error(resp.text)
+             return PostResult(success=False, message=resp.text[:200])
+
+         post_id = resp.json().get("id", "")

```